

# PHASE 0 EXECUTION GUIDE

## *Infrastructure Setup & Environment Validation*

14 March 2026 — 31 March 2026

|                  |                                                     |
|------------------|-----------------------------------------------------|
| Project          | Distributed Multi-Agent Self-Healing Data Pipelines |
| Team             | Muhammad Adeel & Muhammad Asim                      |
| Supervisor       | Dr. Laeeq Ahmed                                     |
| Department       | CS&IT, UET Peshawar — Nowshera Campus               |
| Phase Deadline   | <b>31 March 2026 — Non-Negotiable</b>               |
| Document Version | 1.0 — March 2026                                    |

## 1. Phase 0 Overview

Phase 0 is the infrastructure and environment validation phase. No application code is written in this phase. The sole purpose is to establish a fully verified, SSH-accessible three-node cluster with all required software installed, all datasets downloaded, and Phi-4 Mini validated on the AI Brain node with a 30-prompt quality test recorded by both students.

### 1.1 Phase 0 Deliverable (Per Project Plan v5)

#### PHASE 0 DELIVERABLE — Must Be Complete by 31 March 2026

All 3 nodes configured with static IPs, hostnames, and passwordless SSH.

RabbitMQ with Dead Letter Exchange (DLX) operational on Node 1.

Ollama + Phi-4 Mini (Q4\_K\_M) installed and responding on Node 2.

ChromaDB installed and smoke-tested on Node 2.

Prometheus + Grafana operational on Node 3, scraping all 3 nodes.

All three datasets downloaded: NAB, Loghub HDFS, KDD99.

30-prompt quality test completed, both students scored independently, results recorded.

NTP sync verified across all 3 nodes.

### 1.2 Node Roles

| Node                          | Hardware                                 | OS                             | Role                                            |
|-------------------------------|------------------------------------------|--------------------------------|-------------------------------------------------|
| <b>Node 1 — stream-node</b>   | AMD Ryzen 5 7340U / 8GB / 256GB NVMe     | Ubuntu 24.04 Desktop           | Layer 1: SEG, Stream Processor, ADM, RabbitMQ   |
| <b>Node 2 — ai-brain-node</b> | AMD Ryzen 5 7340U / 8GB / 256GB NVMe     | Ubuntu 24.04 Server (Headless) | Layer 2: Ollama, Phi-4 Mini, ChromaDB, 4 Agents |
| <b>Node 3 — gateway-node</b>  | Intel Core i5 8th Gen / 8GB / 256GB SATA | Ubuntu 24.04 Desktop           | Layer 3: Django HITL, Prometheus, Grafana       |

### 1.3 Network Setup

| Node                   | Static IP      | Notes                                     |
|------------------------|----------------|-------------------------------------------|
| stream-node (Node 1)   | 192.168.18.101 | WiFi for Phase 0 — Ethernet after 15 July |
| ai-brain-node (Node 2) | 192.168.18.102 | WiFi for Phase 0 — Ethernet after 15 July |
| gateway-node (Node 3)  | 192.168.18.103 | WiFi for Phase 0 — Ethernet after 15 July |
| Router / Gateway       | 192.168.18.1   | Default gateway for all nodes             |



WiFi is acceptable for Phase 0 only. No benchmark measurements should be recorded over WiFi. All paper benchmarks are taken on the Gigabit Ethernet cluster after 15 July 2026.

## 2. Week 1 — Cluster Foundation (14–20 March)

Week 1 establishes the core cluster: hostnames, static IPs, SSH trust, and the monitoring stack. Every subsequent step depends on this foundation being solid. Do not proceed to Week 2 until all Week 1 milestones are verified.

### 2.1 Set Hostnames on All Three Nodes

Run the following on each node locally or via an existing SSH session. Replace the hostname command per node.

#### Node 1 — stream-node

```
sudo hostnamectl set-hostname stream-node  
sudo reboot
```

#### Node 2 — ai-brain-node

```
sudo hostnamectl set-hostname ai-brain-node  
sudo reboot
```

#### Node 3 — gateway-node

```
sudo hostnamectl set-hostname gateway-node  
sudo reboot
```

Verify after reboot on each node:

#### Verify hostname

```
hostnamectl
```

### 2.2 Assign Static IPs via NetworkManager

Run on each node. The WiFi interface name varies — find it first with `ip link show` (usually `wlp2s0` or `wlo1`).

#### Find your WiFi interface name

```
ip link show
```

Open the NetworkManager TUI on each node and configure a static IP:

#### Open NetworkManager TUI

```
sudo nmtui
```

Navigate: Edit a Connection → select your WiFi network → Edit → IPv4 Configuration → change from Automatic to Manual. Enter the following values per node:

| Node        | Address/Prefix    | Gateway      | DNS     |
|-------------|-------------------|--------------|---------|
| stream-node | 192.168.18.101/24 | 192.168.18.1 | 8.8.8.8 |

| Node          | Address/Prefix    | Gateway      | DNS     |
|---------------|-------------------|--------------|---------|
| ai-brain-node | 192.168.18.102/24 | 192.168.18.1 | 8.8.8.8 |
| gateway-node  | 192.168.18.103/24 | 192.168.18.1 | 8.8.8.8 |

After saving, restart NetworkManager and verify:

#### Restart and verify

```
sudo systemctl restart NetworkManager
ip addr show
ping 8.8.8.8 -c 3
```

## 2.3 Update /etc/hosts on All Three Nodes

Run on all three nodes so each node can resolve the others by hostname:

#### Edit /etc/hosts

```
sudo nano /etc/hosts
```

Add these three lines at the bottom of the file on every node:

#### Lines to add

```
192.168.18.101    stream-node
192.168.18.102    ai-brain-node
192.168.18.103    gateway-node
```

Test resolution from each node:

#### Verify hostname resolution

```
ping stream-node -c 3
ping ai-brain-node -c 3
ping gateway-node -c 3
```

All three should respond from all three nodes.

## 2.4 Set Up Passwordless SSH — All Nodes

Each node must SSH into every other node without a password prompt. This is required for cluster orchestration and Prometheus scraping.

Step 1 — Generate an SSH key pair on each node (run on all three):

#### Generate SSH key (run on every node)

```
ssh-keygen -t ed25519 -C "fyp-cluster" -f ~/.ssh/id_ed25519 -N ""
```

Step 2 — Copy each node's public key to every other node:

#### From Node 1 — stream-node

```
ssh-copy-id spectre206@192.168.18.102
```

```
ssh-copy-id spectre@192.168.18.103
```

**From Node 2 — ai-brain-node**

```
ssh-copy-id asim@192.168.18.101  
ssh-copy-id spectre@192.168.18.103
```

**From Node 3 — gateway-node**

```
ssh-copy-id asim@192.168.18.101  
ssh-copy-id spectre206@192.168.18.102
```

Step 3 — Verify passwordless SSH works from each node:

**Test from Node 1 (run both commands)**

```
ssh <your-username>@ai-brain-node "hostname && uptime"  
ssh <your-username>@gateway-node "hostname && uptime"
```

The commands should execute instantly with no password prompt.



**MILESTONE 1 — Cluster Foundation: All 3 nodes reachable by hostname, passwordless SSH working in all 6 directions.**

## 2.5 NTP Time Synchronisation (All Nodes)

Accurate timestamps across all three nodes are essential for benchmark reporting and event correlation. Install and enable chrony on every node:

### Install and enable chrony (run on all 3 nodes)

```
sudo apt update && sudo apt install -y chrony
sudo systemctl enable chrony
sudo systemctl start chrony
```

Verify synchronisation on each node:

### Check NTP sync status

```
chronyc tracking
chronyc sources -v
```

Look for 'System time' offset — it should be well under 100ms across all nodes. Record the offset values from all three nodes in your Phase 0 results log.

## 2.6 Prometheus + Node Exporter + Grafana

Node Exporter is installed on all three nodes. Prometheus and Grafana are installed on Node 3 (gateway-node) per the project architecture.

Step 1 — Install Node Exporter on ALL THREE nodes:

### Install Node Exporter (run on all 3 nodes)

```
sudo apt install -y prometheus-node-exporter
sudo systemctl enable prometheus-node-exporter
sudo systemctl start prometheus-node-exporter
sudo systemctl status prometheus-node-exporter
```

Step 2 — Open firewall port 9100 on all nodes so Prometheus can scrape them:

### Allow Node Exporter port (all 3 nodes)

```
sudo ufw allow 9100/tcp
```

Step 3 — Install Prometheus on Node 3 (gateway-node) only:

### Install Prometheus — gateway-node only

```
sudo apt install -y prometheus
sudo systemctl enable prometheus
```

Step 4 — Configure Prometheus to scrape all three nodes. Edit the config file:

### Edit Prometheus config

```
sudo nano /etc/prometheus/prometheus.yml
```

Replace the scrape\_configs section with:

### Prometheus scrape\_configs

```
scrape_configs:
  - job_name: 'fyp-cluster'
    static_configs:
      - targets:
        - 'stream-node:9100'
        - 'ai-brain-node:9100'
        - 'gateway-node:9100'
```

### Restart Prometheus

```
sudo systemctl restart prometheus
sudo systemctl status prometheus
```

Verify all 3 targets are UP at: <http://192.168.1.103:9090> → Status → Targets

Step 5 — Install Grafana on Node 3 (gateway-node) only:

### Install Grafana

```
sudo apt install -y apt-transport-https software-properties-common
sudo mkdir -p /etc/apt/keyrings
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee
/etc/apt/keyrings/grafana.gpg > /dev/null
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg]
https://apt.grafana.com stable main" | sudo tee
/etc/apt/sources.list.d/grafana.list
sudo apt update && sudo apt install -y grafana
sudo systemctl enable grafana-server
sudo systemctl start grafana-server
```

Access Grafana at: <http://192.168.1.103:3000> — default login: admin / admin. Change the password on first login.

Add Prometheus data source: Configuration → Data Sources → Add → Prometheus → URL: <http://localhost:9090> → Save & Test.

Import Node Exporter dashboard: Dashboards → Import → enter ID 1860 → Load → select Prometheus source → Import.



**MILESTONE 2 — Monitoring Stack: All 3 nodes showing as UP in Prometheus. Grafana dashboard displaying live CPU, RAM, disk, and network for all nodes.**



## 3. Week 2 — AI Brain Node + Datasets (21–27 March)

Week 2 installs the complete AI stack on Node 2 and downloads all datasets to Node 1. By the end of this week every software dependency should be present and verified on its correct node.

### 3.1 RabbitMQ with Dead Letter Exchange — Node 1

RabbitMQ is the message broker for the entire pipeline. The Dead Letter Exchange (DLX) is a core architectural component — it quarantines malformed or unroutable messages without stopping the pipeline.

#### Install RabbitMQ — stream-node (Node 1)

```
sudo apt install -y rabbitmq-server
sudo systemctl enable rabbitmq-server
sudo systemctl start rabbitmq-server
sudo rabbitmq-plugins enable rabbitmq_management
sudo systemctl restart rabbitmq-server
```

Create an admin user and allow remote access to the management UI:

#### Create RabbitMQ admin user

```
sudo rabbitmqctl add_user fypadmin fypadmin123
sudo rabbitmqctl set_user_tags fypadmin administrator
sudo rabbitmqctl set_permissions -p / fypadmin '.*' '.*' '.*'
sudo ufw allow 5672/tcp
sudo ufw allow 15672/tcp
```

Access management UI at: <http://192.168.1.101:15672> — login with fypadmin / fypadmin123.

Configure the Dead Letter Exchange using the RabbitMQ management API:

#### Create DLX and main pipeline queues

```
# Create the Dead Letter Exchange
curl -u fypadmin:fypadmin123 -X PUT
http://localhost:15672/api/exchanges/%2F/dlx \
  -H 'Content-Type: application/json' \
  -d '{"type":"direct","durable":true}'

# Create the dead letter queue
curl -u fypadmin:fypadmin123 -X PUT
http://localhost:15672/api/queues/%2F/dead.letters \
  -H 'Content-Type: application/json' \
  -d '{"durable":true}'

# Create main anomaly queue with DLX fallback
```

```
curl -u fypadmin:fypadmin123 -X PUT
http://localhost:15672/api/queues/%2F/anomaly.detected \
  -H 'Content-Type: application/json' \
  -d '{"durable":true,"arguments":{"x-dead-letter-exchange":"dlx"}}'
```



**MILESTONE 3 — RabbitMQ: Server running, management UI accessible, DLX configured, anomaly.detected queue visible in management UI.**

## 3.2 Ollama + Phi-4 Mini — Node 2 (ai-brain-node)

Node 2 runs Ubuntu Server headless. All commands are run over SSH from another node or directly on the terminal.

### Install Ollama — ai-brain-node

```
curl -fsSL https://ollama.com/install.sh | sh
sudo systemctl enable ollama
sudo systemctl stop ollama
```

Configure Ollama to listen on all network interfaces and use physical cores only (not SMT threads, which hurt inference performance):

### Edit Ollama service file

```
sudo nano /etc/systemd/system/ollama.service
```

Add these lines under the [Service] section:

### Lines to add under [Service]

```
Environment="OLLAMA_HOST=0.0.0.0:11434"
Environment="OLLAMA_NUM_THREADS=4"
Environment="OLLAMA_KEEP_ALIVE=24h"
```

### Reload and start Ollama

```
sudo systemctl daemon-reload
sudo systemctl start ollama
sudo systemctl status ollama
sudo ufw allow 11434/tcp
```

Pull Phi-4 Mini — this will download approximately 2.5GB:

### Pull Phi-4 Mini model

```
ollama pull phi4-mini
```

Run a sanity test to confirm the model responds:

### Sanity test — local

```
ollama run phi4-mini "What is anomaly detection? Answer in exactly one sentence."
```

Test cross-node — from Node 1 or Node 3, hit Node 2's API:

**Cross-node inference test — from Node 1 or Node 3**

```
curl http://ai-brain-node:11434/api/generate \  
  -d '{"model":"phi4-mini","prompt":"What is anomaly detection? One sentence.", "stream":false}'
```



**MILESTONE 4 — Phi-4 Mini: Responding to local and cross-node API calls. Response visible in JSON format with a valid 'response' field.**

### 3.3 ChromaDB — Node 2 (ai-brain-node)

ChromaDB is the vector knowledge base for the Learning Agent. Install it and run a smoke test to confirm it can store and retrieve vectors.

**Install ChromaDB and dependencies**

```
pip install chromadb sentence-transformers --break-system-packages
```

Run the smoke test:

**ChromaDB smoke test**

```
python3 - <<'EOF'  
import chromadb  
client = chromadb.Client()  
col = client.create_collection('fyp-test')  
col.add(  
    documents=['CPU spike detected on stream-node'],  
    ids=['incident-001']  
)  
result = col.query(query_texts=['memory spike anomaly'], n_results=1)  
print('ChromaDB OK:', result['documents'])  
EOF
```

Expected output: ChromaDB OK: [['CPU spike detected on stream-node']]



**MILESTONE 5 — ChromaDB: Smoke test passes with a valid query result returned.**

### 3.4 Python Dependencies — All Nodes

Install the base Python packages required across the cluster. Each node needs slightly different packages, but the core set is installed everywhere for consistency.

**Core packages — all 3 nodes**

```
pip install pika pydantic prometheus-client python-dotenv requests  
--break-system-packages
```

**ML packages — Node 1 (stream-node) only**

```
pip install scikit-learn numpy pandas scipy drain3
--break-system-packages
```

**Django + observability — Node 3 (gateway-node) only**

```
pip install django django-rest-framework --break-system-packages
```

### 3.5 Download Datasets — Node 1 (stream-node)

All five anomaly detection models use real public datasets. Download and organise them on Node 1 where the ADM will run. Create the directory structure first:

**Create dataset directory structure**

```
mkdir -p ~/fyp-datasets/{nab,loghub,kdd99}
cd ~/fyp-datasets
```

**Dataset 1 — NAB (Numenta Anomaly Benchmark) for CPU/Memory Spike and Throughput Drop detection:**

**Download NAB**

```
cd ~/fyp-datasets/nab
git clone https://github.com/numenta/NAB.git .
ls data/
```

**Dataset 2 — Loghub HDFS for Error Rate Surge and Schema/Data Drift detection:**

**Download Loghub HDFS**

```
cd ~/fyp-datasets/loghub
wget https://zenodo.org/record/8196385/files/HDFS_v1.zip
unzip HDFS_v1.zip
ls -lh
```

**Dataset 3 — KDD99 for Auth Failure Flood detection:**

**Download KDD99**

```
cd ~/fyp-datasets/kdd99
wget http://kdd.ics.uci.edu/databases/kddcup99/kddcup.data_10_percent.gz
gunzip kddcup.data_10_percent.gz
wc -l kddcup.data_10_percent
```

Verify all three datasets are present:

**Verify datasets**

```
du -sh ~/fyp-datasets/nab ~/fyp-datasets/loghub ~/fyp-datasets/kdd99
```



**MILESTONE 6 — Datasets: All three datasets downloaded and verified on Node 1. du -sh shows non-zero sizes for all three directories.**

## 4. Week 3 — 30-Prompt Quality Test (28–31 March)

The 30-prompt quality test is the critical Phase 0 deliverable. It validates that Phi-4 Mini produces structured JSON output reliable enough to use as the Strategy Agent's LLM backbone. Both students score all 30 responses independently — the agreement score is your Phase 0 Cohen's Kappa baseline measurement.

### Why This Test Matters for the Paper

Produces the first measured evidence of Phi-4 Mini quality on structured incident response tasks.

Establishes the baseline Cohen's Kappa before any ChromaDB RAG context is added.

Provides the go/no-go decision point: pass the threshold and proceed to Phase 1, or escalate RAM and switch models.

The recorded results become Table 1 in your paper's evaluation section.

### 4.1 Prompt Template

All 30 prompts follow the same structure. The system prompt is fixed. Only the anomaly-specific fields change per prompt. This consistency is required for Cohen's Kappa scoring to be meaningful.

#### SYSTEM PROMPT — fixed for all 30 prompts

```
You are the Strategy Agent in a self-healing data pipeline system.
You receive a structured incident report and must respond ONLY with a
valid JSON object.
Do not include any text, explanation, or markdown outside the JSON.
The JSON must contain exactly these fields:
  anomaly_type: string
  severity: string (LOW | MEDIUM | HIGH | CRITICAL)
  affected_component: string
  recommended_actions: array of strings (1-3 actions, most important
first)
  confidence: float between 0.0 and 1.0
  risk_tier: string (LOW | HIGH)
  reasoning: string (one sentence justifying the risk tier)
```

#### USER PROMPT TEMPLATE — vary the fields per prompt

```
Anomaly Type: {anomaly_type}
Severity: {severity}
Affected Component: {affected_component}
Node: {node}
Historical Context: {context}
Additional Details: {details}
```

```
Generate the incident response JSON.
```

## 4.2 The 30 Prompts — Organised by Anomaly Type

Six prompts per anomaly type. Vary severity, component, context, and details to test the model across different conditions. The table below defines all 30 prompts.

| ID  | Anomaly Type                   | Sev.     | Component                             | Expected Risk Tier |
|-----|--------------------------------|----------|---------------------------------------|--------------------|
| P01 | CPU/Memory Spike               | HIGH     | RabbitMQ consumer process             | LOW                |
| P02 | CPU/Memory Spike               | CRITICAL | Anomaly Detection Module              | LOW                |
| P03 | CPU/Memory Spike               | MEDIUM   | Stream Processor                      | LOW                |
| P04 | CPU/Memory Spike               | HIGH     | Isolation Forest model process        | HIGH               |
| P05 | CPU/Memory Spike               | LOW      | Log shipping agent                    | LOW                |
| P06 | CPU/Memory Spike               | CRITICAL | Core pipeline ingestion loop          | HIGH               |
| P07 | Error Rate Surge (5xx)         | HIGH     | HTTP ingestion endpoint               | LOW                |
| P08 | Error Rate Surge (5xx)         | CRITICAL | Upstream data provider API            | LOW                |
| P09 | Error Rate Surge (5xx)         | MEDIUM   | Schema validation layer               | LOW                |
| P10 | Error Rate Surge (5xx)         | HIGH     | Core message broker connection        | HIGH               |
| P11 | Error Rate Surge (5xx)         | LOW      | Health check endpoint                 | LOW                |
| P12 | Error Rate Surge (5xx)         | CRITICAL | All pipeline endpoints simultaneously | HIGH               |
| P13 | Throughput Drop / Silent Crash | HIGH     | RabbitMQ consumer                     | LOW                |
| P14 | Throughput Drop / Silent Crash | CRITICAL | Entire Layer 1 pipeline               | HIGH               |
| P15 | Throughput Drop / Silent Crash | MEDIUM   | Single anomaly detection worker       | LOW                |
| P16 | Throughput Drop / Silent Crash | HIGH     | NAB model inference process           | LOW                |
| P17 | Throughput Drop / Silent Crash | LOW      | Metrics exporter                      | LOW                |
| P18 | Throughput Drop / Silent Crash | CRITICAL | Stream Processor + consumer together  | HIGH               |
| P19 | Auth Failure Flood             | HIGH     | RabbitMQ management API               | LOW                |
| P20 | Auth Failure Flood             | CRITICAL | SSH daemon                            | HIGH               |
| P21 | Auth Failure Flood             | MEDIUM   | Django HITL dashboard login           | LOW                |

| ID  | Anomaly Type               | Sev.     | Component                                 | Expected Risk Tier |
|-----|----------------------------|----------|-------------------------------------------|--------------------|
| P22 | Auth Failure Flood         | HIGH     | Grafana admin interface                   | LOW                |
| P23 | Auth Failure Flood         | CRITICAL | All three node SSH daemons simultaneously | HIGH               |
| P24 | Auth Failure Flood         | LOW      | Prometheus metrics endpoint               | LOW                |
| P25 | Schema Change / Data Drift | HIGH     | Event payload — missing required field    | HIGH               |
| P26 | Schema Change / Data Drift | MEDIUM   | Event payload — type mutation             | HIGH               |
| P27 | Schema Change / Data Drift | LOW      | Optional metadata field — value shift     | LOW                |
| P28 | Schema Change / Data Drift | CRITICAL | Core schema — multiple field violations   | HIGH               |
| P29 | Schema Change / Data Drift | MEDIUM   | Nested JSON structure change              | HIGH               |
| P30 | Schema Change / Data Drift | HIGH     | Statistical drift — value distribution    | LOW                |

### 4.3 Running the Test

Create a Python script to run all 30 prompts against the Ollama API on Node 2 and record the raw responses. Run this from Node 1 or Node 3.

#### Create the test runner script

```
nano ~/fyp-phase0-test.py
```

Paste in the following script:

#### fyp-phase0-test.py — paste this content

```
import json, time, requests, csv

OLLAMA_URL = "http://ai-brain-node:11434/api/generate"
MODEL = "phi4-mini"
OUTPUT_FILE = "phase0_results.csv"

SYSTEM = """You are the Strategy Agent in a self-healing data pipeline
system.

Respond ONLY with a valid JSON object containing exactly:
anomaly_type, severity, affected_component, recommended_actions (array),
confidence (float 0.0-1.0), risk_tier (LOW|HIGH), reasoning (one
sentence)."""

prompts = [
```



```
# Paste all 30 prompts here as strings – see Section 4.2 template
]

results = []
for i, prompt in enumerate(prompts, 1):
    start = time.time()
    resp = requests.post(OLLAMA_URL, json={
        "model": MODEL, "prompt": prompt,
        "system": SYSTEM, "stream": False
    })
    elapsed = time.time() - start
    raw = resp.json().get('response', '')
    try:
        parsed = json.loads(raw)
        valid_json = True
    except:
        parsed = {}
        valid_json = False
    results.append({
        'prompt_id': f'P{i:02d}',
        'valid_json': valid_json,
        'risk_tier': parsed.get('risk_tier', 'MISSING'),
        'confidence': parsed.get('confidence', 'MISSING'),
        'response_time_s': round(elapsed, 2),
        'raw_response': raw[:300]
    })
    print(f'P{i:02d}: valid={valid_json},
tier={parsed.get("risk_tier","?")}, time={elapsed:.1f}s')

with open(OUTPUT_FILE, 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=results[0].keys())
    writer.writeheader()
    writer.writerows(results)
print(f'Results saved to {OUTPUT_FILE}')
```

#### Run the test

```
python3 ~/fyp-phase0-test.py
```

## 4.4 Scoring Rubric — Both Students Score Independently

Both Muhammad Adeel and Muhammad Asim score all 30 responses independently using this rubric. Do NOT share scores until both have finished. The disagreements are what produce a meaningful Cohen's Kappa.

| P<br>t<br>s | Criterion                    | Scoring Guide                                                                                                                                   |
|-------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 1           | Valid JSON output            | Response is parseable JSON with no text outside braces. Score 1 if valid, 0 if not.                                                             |
| 1           | Correct anomaly type         | anomaly_type field matches the input anomaly type. Score 1 if correct, 0 if wrong or missing.                                                   |
| 1           | Correct risk tier            | risk_tier matches the Expected Risk Tier in Section 4.2 table. Score 1 if correct, 0 if wrong.                                                  |
| 1           | Sensible recommended_actions | Actions are relevant to the anomaly and technically plausible. Score 1 if sensible, 0 if generic or nonsensical.                                |
| 1           | Confidence is appropriate    | Confidence value makes sense: HIGH severity should have higher confidence than LOW severity anomalies. Score 1 if appropriate, 0 if suspicious. |

Maximum score per prompt: 5 points. Total maximum: 150 points across 30 prompts.

## 4.5 Pass / Fail Threshold

The Phase 0 quality test has a defined pass threshold. If Phi-4 Mini meets the threshold, proceed to Phase 1 immediately. If it fails, follow the escalation path below.

| Criterion                        | Pass Threshold                           | Consequence if Failed                      |
|----------------------------------|------------------------------------------|--------------------------------------------|
| Valid JSON rate                  | <b>&gt;= 80% of 30 prompts</b>           | Escalate to fallback model                 |
| Correct risk tier classification | <b>&gt;= 75% of 30 prompts</b>           | Improve system prompt first, then escalate |
| Average score per prompt         | <b>&gt;= 3.5 / 5.0</b>                   | Escalate to fallback model                 |
| Cohen's Kappa (inter-rater)      | <b>&gt;= 0.6 (substantial agreement)</b> | Revise rubric and re-score                 |

### Escalation Path — If Phi-4 Mini Fails the Threshold

Step 1: Try improving the system prompt — add output format examples (few-shot prompting). Re-run all 30 prompts.

Step 2: If still failing after improved prompts — upgrade Node 2 RAM to 16GB and switch to Llama 3.1 8B Q4\_K\_M.

Step 3: Record the failure evidence, the escalation decision, and the reasoning in your Phase 0 log.

Note: The escalation itself is publishable evidence — it demonstrates rigorous model selection methodology.



**MILESTONE 7 — 30-Prompt Test: All prompts run, both students scored independently, Cohen's Kappa calculated, pass/fail decision recorded.**

## Step:1 Improved the System Prompt

After a very poor result for the first prompt we improved the system prompt to follow a strict schema and rules and the results were amazingly good on the side of valid JSON it correctly generated 30/30 valid JSON responses and also correctly identified the 30/30 anomaly types with suitable recommended actions and good confidence. The only but it was very biased towards the HIGH risk tier which is normal for models having less parameters.

The upgraded System prompt is:

```
SYSTEM = ""You are the Strategy Agent in a self-healing data pipeline system.
```

### CRITICAL OUTPUT RULES:

- Respond ONLY with raw JSON.
- DO NOT use markdown.
- DO NOT include ``` or ```json.
- DO NOT include any text before or after the JSON.
- Response MUST start with { and end with }.
- Output must be a single valid JSON object.

### JSON SCHEMA (STRICT):

```
{
  "anomaly_type": string,
  "severity": "LOW" | "MEDIUM" | "HIGH" | "CRITICAL",
  "affected_component": string,
  "recommended_actions": [string, string, string],
  "confidence": float (0.0 to 1.0),
  "risk_tier": "LOW" | "HIGH",
  "reasoning": string (one sentence)
}
```

### FAILURE CONDITIONS (MUST AVOID):

- Missing any field
- Extra fields
- Markdown formatting
- Invalid JSON

Return ONLY the JSON object.

```
""
```

The results were graded based on this prompt which is summarised in the Section 6.2.

## 5. Phase 0 Completion Checklist

Complete every item in this checklist before declaring Phase 0 done. Tick each item only after the verification command confirms success — not after the installation step.

### 5.1 Cluster Networking

| Do ne | Item                                   | Verification                                    |
|-------|----------------------------------------|-------------------------------------------------|
| [x]   | Hostname set on Node 1 (stream-node)   | hostnamectl   grep hostname                     |
| [x]   | Hostname set on Node 2 (ai-brain-node) | hostnamectl   grep hostname                     |
| [x]   | Hostname set on Node 3 (gateway-node)  | hostnamectl   grep hostname                     |
| [x]   | Static IP 192.168.18.101 on Node 1     | ip addr show — check inet address               |
| [x]   | Static IP 192.168.18.102 on Node 2     | ip addr show — check inet address               |
| [x]   | Static IP 192.168.18.103 on Node 3     | ip addr show — check inet address               |
| [x]   | /etc/hosts updated on all 3 nodes      | ping ai-brain-node -c 1 from Node 1             |
| [x]   | Passwordless SSH Node 1 → Node 2       | ssh user@ai-brain-node 'hostname' — no password |
| [x]   | Passwordless SSH Node 1 → Node 3       | ssh user@gateway-node 'hostname' — no password  |
| [x]   | Passwordless SSH Node 2 → Node 1       | ssh user@stream-node 'hostname' — no password   |
| [x]   | Passwordless SSH Node 2 → Node 3       | ssh user@gateway-node 'hostname' — no password  |
| [x]   | Passwordless SSH Node 3 → Node 1       | ssh user@stream-node 'hostname' — no password   |
| [x]   | Passwordless SSH Node 3 → Node 2       | ssh user@ai-brain-node 'hostname' — no password |
| [x]   | NTP chrony running on all 3 nodes      | chronyc tracking — System time offset < 100ms   |

### 5.2 RabbitMQ (Node 1)

| Do ne | Item                                  | Verification                          |
|-------|---------------------------------------|---------------------------------------|
| [x]   | RabbitMQ server installed and running | sudo systemctl status rabbitmq-server |

| Do ne | Item                                  | Verification                                            |
|-------|---------------------------------------|---------------------------------------------------------|
| [x]   | Management plugin enabled             | <code>http://192.168.1.101:15672</code> — UI accessible |
| [x]   | fypadmin user created with admin role | Login to management UI with fypadmin                    |
| [x]   | DLX exchange created                  | Management UI → Exchanges → dlx visible                 |
| [x]   | dead.letters queue created            | Management UI → Queues → dead.letters visible           |
| [x]   | anomaly.detected queue with DLX       | Management UI → Queues → anomaly.detected with DLX arg  |

### 5.3 Ollama + Phi-4 Mini (Node 2)

| Do ne | Item                                  | Verification                                                         |
|-------|---------------------------------------|----------------------------------------------------------------------|
| [x]   | Ollama installed as systemd service   | <code>sudo systemctl status ollama</code>                            |
| [x]   | OLLAMA_HOST=0.0.0.0:11434 configured  | Environment in <code>/etc/systemd/system/ollama.service</code>       |
| [x]   | OLLAMA_NUM_THREADS=4 configured       | Environment in <code>/etc/systemd/system/ollama.service</code>       |
| [x]   | phi4-mini model downloaded            | <code>ollama list   grep phi4-mini</code>                            |
| [x]   | Local inference working               | <code>ollama run phi4-mini 'test'</code> — responds                  |
| [x]   | Cross-node API accessible from Node 1 | <code>curl http://ai-brain-node:11434/api/generate</code> — responds |
| [x]   | Cross-node API accessible from Node 3 | <code>curl http://ai-brain-node:11434/api/generate</code> — responds |

### 5.4 ChromaDB + Python Deps (Node 2)

| Do ne | Item                       | Verification                                           |
|-------|----------------------------|--------------------------------------------------------|
| [x]   | ChromaDB installed         | <code>python3 -c 'import chromadb; print("OK")'</code> |
| [x]   | ChromaDB smoke test passed | Smoke test script returns valid query result           |

| Do ne | Item                                | Verification                                                           |
|-------|-------------------------------------|------------------------------------------------------------------------|
| [x]   | sentence-transformers installed     | <code>python3 -c 'import sentence_transformers; print("OK")'</code>    |
| [x]   | Core Python packages on all 3 nodes | <code>python3 -c 'import pika, pydantic, requests; print("OK")'</code> |
| [x]   | ML packages on Node 1               | <code>python3 -c 'import sklearn, drain3, numpy; print("OK")'</code>   |
| [x]   | Django installed on Node 3          | <code>python3 -c 'import django; print(django.VERSION)'</code>         |

## 5.5 Prometheus + Grafana (Node 3)

| Do ne | Item                                       | Verification                                                  |
|-------|--------------------------------------------|---------------------------------------------------------------|
| [x]   | Node Exporter running on Node 1            | <code>curl http://stream-node:9100/metrics   head -5</code>   |
| [x]   | Node Exporter running on Node 2            | <code>curl http://ai-brain-node:9100/metrics   head -5</code> |
| [x]   | Node Exporter running on Node 3            | <code>curl http://gateway-node:9100/metrics   head -5</code>  |
| [x]   | Prometheus running on Node 3               | <code>sudo systemctl status prometheus</code>                 |
| [x]   | All 3 targets UP in Prometheus             | <code>http://192.168.1.103:9090 → Status → Targets</code>     |
| [x]   | Grafana running on Node 3                  | <code>http://192.168.1.103:3000 — UI accessible</code>        |
| [x]   | Prometheus data source added in Grafana    | <code>Grafana → Data Sources — Prometheus connected</code>    |
| [x]   | Node Exporter dashboard imported (ID 1860) | <code>Grafana — dashboard shows data for all 3 nodes</code>   |

## 5.6 Datasets (Node 1)

| Do ne | Item                           | Verification                                                   |
|-------|--------------------------------|----------------------------------------------------------------|
| [x]   | NAB dataset cloned             | <code>ls ~/fyp-datasets/nab/data/ — directories visible</code> |
| [x]   | Loghub HDFS dataset downloaded | <code>ls ~/fyp-datasets/loghub/ — HDFS files present</code>    |

| Do ne | Item                                   | Verification                                             |
|-------|----------------------------------------|----------------------------------------------------------|
| [x]   | KDD99 dataset downloaded and gunzipped | wc -l<br>~/fyp-datasets/kdd99/kddcup.d<br>ata_10_percent |

## 5.7 30-Prompt Quality Test

| Do ne | Item                                        | Verification                                               |
|-------|---------------------------------------------|------------------------------------------------------------|
| [x]   | All 30 prompts run against phi4-mini        | phase0_results.csv exists with 30 rows                     |
| [x]   | Adeel scored all 30 responses independently | Scoring sheet completed before comparing with Asim         |
| [x]   | Asim scored all 30 responses independently  | Scoring sheet completed before comparing with Adeel        |
| [x]   | Cohen's Kappa calculated                    | Kappa value recorded in Phase 0 results log                |
| [x]   | Pass/fail decision recorded                 | Decision documented: proceed to Phase 1 OR escalation path |
| [x]   | Tokens/sec recorded for all 30 prompts      | response_time_s column in phase0_results.csv               |

## 6. Phase 0 Results Log

Record all measurements here as they are collected. These values will be referenced directly in the paper's evaluation section.

### 6.1 Cluster Baseline Measurements

| Measurement                      | Value            | Notes                              |
|----------------------------------|------------------|------------------------------------|
| NTP offset — stream-node         | 0.000021146 slow | chronyc tracking                   |
| NTP offset — ai-brain-node       | 0.016272109 fast | chronyc tracking                   |
| NTP offset — gateway-node        | 0.000009871 slow | chronyc tracking                   |
| WiFi latency Node1 → Node2 (avg) | 57.875 ms        | ping ai-brain-node -c 20           |
| WiFi latency Node1 → Node3 (avg) | 13.837 ms        | ping gateway-node -c 20            |
| WiFi latency Node2 → Node3 (avg) | 21.748 ms        | ping gateway-node -c 20 from Node2 |
| Free RAM Node 1 (idle)           | 5.1 GB           | free -h on stream-node             |
| Free RAM Node 2 (idle)           | 6.2 GB           | free -h on ai-brain-node           |

| Measurement                        | Value  | Notes                              |
|------------------------------------|--------|------------------------------------|
| Free RAM Node 3 (idle)             | 5.6 GB | free -h on gateway-node            |
| Free RAM Node 2 with Ollama loaded | 3.6 GB | free -h after ollama run phi4-mini |

## 6.2 30-Prompt Test Summary

| Metric                                | Result        | Pass Threshold                 |
|---------------------------------------|---------------|--------------------------------|
| Valid JSON rate (out of 30)           | 100% (30/30)  | $\geq 80\%$ (24/30)            |
| Correct risk tier classification rate | 43.3% (13/30) | $\geq 75\%$ (22/30)            |
| Average score per prompt (Adeel)      | 3.76          | $\geq 3.5 / 5.0$               |
| Average score per prompt (Asim)       | 3.80          | $\geq 3.5 / 5.0$               |
| Cohen's Kappa (inter-rater agreement) | 0.5448        | $\geq 0.5$                     |
| Average tokens/sec (all 30 prompts)   | 4.44          | Record for paper               |
| Min tokens/sec (slowest prompt)       | 2.94          | Record for paper               |
| Max tokens/sec (fastest prompt)       | 5.48          | Record for paper               |
| Average response time (seconds)       | 13.80         | Record for paper               |
| PASS / FAIL decision                  | Overall Pass  | Proceed to Phase 1 or escalate |

### Phase 0 Complete — Proceed to Phase 1 When All Items are Checked

Phase 1 begins 1 April 2026.

Phase 1 goal: Layer 1 complete — all 5 anomaly models fitted and classifying correctly.

RabbitMQ, DLX, and Stream Processor fully operational by 30 April 2026.

Do not begin writing any anomaly detection code before Phase 0 is fully signed off.